
spaCy
Text
nlp.vocab
Vocab
StringStore
Vectors
nlp
Language
Config & Meta
Language data
CONTAINS
CREATES
nlp.tokenizer
Tokenizer
nlp.pipeline
Component
Component
Model
Weights
doc[i].lex
Lexeme
CREATES
PROCESSES
TRAINS
Doc
CREATES
CREATES
Example
Doc
Doc
ACCESSES
doc[i]
Token
doc[a:b]
Span
Data

INPUT

```
import spacy

nlp = spacy.blank("en")
doc = nlp("I have a cat")

# Look up the hash for the word "cat"
cat_hash = nlp.vocab.strings["cat"]
print(cat_hash)

# Look up the cat_hash to get the string
cat_string = nlp.vocab.strings[cat_hash]
print(cat_string)
```

Run code Submit

OUTPUT

```
5439657043933447811
cat
✓ Great work!
```

INPUT

```
import spacy

nlp = spacy.blank("en")
doc = nlp("David Bowie is a PERSON")

# Look up the hash for the string label "PERSON"
person_hash = nlp.vocab.strings["PERSON"]
print(person_hash)

# Look up the person_hash to get the string
person_string = nlp.vocab.strings[person_hash]
print(person_string)
```

Run code Submit

OUTPUT

```
380
PERSON
✓ Good job!
```

Why does this code throw an error?

```
import spacy

# Create an English and German nlp object
nlp = spacy.blank("en")
nlp_de = spacy.blank("de")

# Get the ID for the string 'Bowie'
bowie_id = nlp.vocab.strings["Bowie"]
print(bowie_id)

# Look up the ID for "Bowie" in the vocab
print(nlp_de.vocab.strings[bowie_id])
```

- (●) The string `"Bowie"` isn't in the German vocab, so the hash can't be resolved in the string store.
- ( ) `"Bowie"` is not a regular word in the English or German dictionary, so it can't be hashed.
- ( ) `nlp_de` is not a valid name. The vocab can only be shared if the `nlp` objects have the same name.

Submit

**That's correct!** Hashes can't be reversed. To prevent this problem, add the word to the new vocab by processing a text or looking up the string, or use the same vocab to resolve the hash back to a string.

INPUT

```
import spacy

nlp = spacy.blank("en")

# Import the Doc class
from spacy.tokens import Doc

# Desired text: "spaCy is cool!"
words = ["spaCy", "is", "cool", "!"]
spaces = [True, True, False, False]

# Create a Doc from the words and spaces
doc = Doc(nlp.vocab, words=words, spaces=spaces)
print(doc.text)
```

Run code Submit

OUTPUT

```
spaCy is cool!
✓ Well done!
```

INPUT

```
import spacy

nlp = spacy.blank("en")

# Import the Doc class
from spacy.tokens import Doc

# Desired text: "Go, get started!"
words = ["Go", ",", "get", "started", "!"]
spaces = [False, True, True, False, False]

# Create a Doc from the words and spaces
doc = Doc(nlp.vocab, words=words, spaces=spaces)
print(doc.text)
```

Run code Submit

OUTPUT

```
Go, get started!
✓ Nice!
```

INPUT

```
import spacy

nlp = spacy.blank("en")

# Import the Doc class
from spacy.tokens import Doc

# Desired text: "Oh, really?!"
words = ['Oh', ',', 'really', '?', '!']
spaces = [False, True, False, False, False]

# Create a Doc from the words and spaces
doc = Doc(nlp.vocab, words, spaces)
print(doc.text)
```

Run code Submit

OUTPUT

```
Oh, really?!
✓ Nice work! Next, let's create some entities.
```

INPUT

```
import spacy

nlp = spacy.blank("en")

# Import the Doc and Span classes
from spacy.tokens import Doc, Span

words = ["I", "like", "David", "Bowie"]
spaces = [True, True, True, False]

# Create a doc from the words and spaces
doc = Doc(nlp.vocab, words, spaces)
print(doc.text)

# Create a span for "David Bowie" from the doc and assign it the label "PERSON"
span = Span(doc, 2, 4, label='PERSON')
print(span.text, span.label_)

# Add the span to the doc's entities
doc.ents = [span]

# Print entities' text and labels
print([(ent.text, ent.label_) for ent in doc.ents])
```

Run code Submit

OUTPUT

```
I like David Bowie
David Bowie PERSON
[('David Bowie', 'PERSON')]
✓ Perfect! Creating spaCy's objects manually and modifying the entities
will come in handy later when you're writing your own information extraction
pipelines.
```

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Berlin looks like a nice city")

# Get all tokens and part-of-speech tags
token_texts = [token.text for token in doc]
pos_tags = [token.pos_ for token in doc]

for index, pos in enumerate(pos_tags):
    # Check if the current token is a proper noun
    if pos == "PROPN":
        # Check if the next token is a verb
        if pos_tags[index + 1] == "VERB":
            result = token_texts[index]
            print("Found proper noun before a verb:", result)
```

**Part 1**

Why is the code bad?

- ◯ The `result` token should be converted back to a `Token` object. This will let you reuse it in spaCy.
- ◉ It only uses lists of strings instead of native token attributes. This is often less efficient, and can't express complex relationships.
- ◯ `pos_` is the wrong attribute to use for extracting proper nouns. You should use `tag_` and the `"NNP"` and `"NNS"` labels instead.

Submit

**That's correct!** Always convert the results to strings as late as possible, and try to use native token attributes to keep things consistent.

INPUT

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Berlin looks like a nice city")

for token in doc:
    # Check if the current token is a proper noun
    if token.pos_ == "PROPN":
        # Check if the next token is a verb
        if doc[token.i + 1].pos_ == "VERB":
            print("Found proper noun before a verb:", token.text)
```

Run code  Submit

OUTPUT

```
Found proper noun before a verb: Berlin
```

✓ Great work! While the solution here works fine for the given example, there are still things that can be improved. If the doc ends with a proper noun, doc[token.i + 1] will fail. To make sure the code generalizes, you should first check if token.i + 1 < len(doc).

```
doc1 = nlp("I like cats")
doc2 = nlp("I hate cats")

print(doc1.similarity(doc2))
```

```
0.9501447503553421
```

INPUT

```
import spacy

# Load the en_core_web_md pipeline
nlp = spacy.load('en_core_web_md')

# Process a text
doc = nlp("Two bananas in pyjamas")

# Get the vector for the token "bananas"
bananas_vector = doc[1].vector # 300-dim
print(bananas_vector)
```

Run code  Submit

OUTPUT

```
[-0.6334     0.18981   -0.53544   -0.52658   -0.30001    0.30559
 -0.49303    0.14636    0.012273   0.96802    0.0040354  0.25234
 -0.29864   -0.014646  -0.24905   -0.67125   -0.053366   0.59426
 -0.068034   0.10315    0.66759    0.024617  -0.37548    0.52557
```

INPUT

```
import spacy

nlp = spacy.load("en_core_web_md")

doc1 = nlp("It's a warm summer day")
doc2 = nlp("It's sunny outside")

# Get the similarity of doc1 and doc2
similarity = doc1.similarity(doc2) # Word2Vec, cosine similarity
print(similarity)
```

Run code  Submit

OUTPUT

```
0.8456854224205017
✓ Well done!
```

INPUT

```
import spacy

nlp = spacy.load("en_core_web_md")

doc = nlp("TV and books")
token1, token2 = doc[0], doc[2]

# Get the similarity of the tokens "TV" and "books"
similarity = token1.similarity(token2)
print(similarity)
```

Run code  Submit

OUTPUT

```
0.18317238986492157
✓ Nicely done!
```

INPUT

```
import spacy

nlp = spacy.load("en_core_web_md")

doc = nlp("This was a great restaurant. Afterwards, we went to a really nice bar.")

# Create spans for "great restaurant" and "really nice bar"
span1 = doc[3:5]
span2 = doc[12:15]

# Get the similarity of the spans
print(span1.text)
print(span2.text)
similarity = span1.similarity(span2)
print(similarity)
```

Run code  Submit

OUTPUT

```
great restaurant
really nice bar
0.7541285157203674
✓ Well done! Feel free to experiment with comparing more objects, if
you like. The similarities are not *always* this conclusive. Once you're getting
serious about developing NLP applications that leverage semantic similarity, you
might want to train vectors on your own data, or tweak the similarity
algorithm.
```

Why does this pattern not match the tokens “Silicon Valley” in the `doc`?

```
pattern = [{"LOWER": "silicon"}, {"TEXT": " "}, {"LOWER": "valley"}]
```

```
doc = nlp("Can Silicon Valley workers rein in big tech from within?")
```

- ○ The tokens "Silicon" and "Valley" are not lowercase, so the `"LOWER"` attribute won’t match.
- ● The tokenizer doesn’t create tokens for single spaces, so there’s no token with the value `" "` in between.
- ○ The tokens are missing an operator `"OP"` to indicate that they should be matched exactly once.

Submit

**That's correct!** The tokenizer already takes care of splitting off whitespace and each dictionary in the pattern describes one token.

INPUT

```
import spacy
from spacy.matcher import Matcher

nlp = spacy.load("en_core_web_sm")
doc = nlp(
    "Twitch Prime, the perks program for Amazon Prime members offering free "
    "loot, games and other benefits, is ditching one of its best features: "
    "ad-free viewing. According to an email sent out to Amazon Prime members "
    "today, ad-free viewing will no longer be included as a part of Twitch "
    "Prime for new members, beginning on September 14. However, members with "
    "existing annual subscriptions will be able to continue to enjoy ad-free "
    "viewing until their subscription comes up for renewal. Those with "
    "monthly subscriptions will have access to ad-free viewing until October 15."
)

# Create the match patterns
pattern1 = [{"LOWER": "amazon"}, {"IS_TITLE": True, "POS": "PROPN"}]
pattern2 = [{"LOWER": "ad"}, {"TEXT": "-"}, {"LOWER": "free"}, {"POS": "NOUN"}]

# Initialize the Matcher and add the patterns
matcher = Matcher(nlp.vocab)
matcher.add("PATTERN1", [pattern1])
matcher.add("PATTERN2", [pattern2])

# Iterate over the matches
for match_id, start, end in matcher(doc):
    # Print pattern string name and text of matched span
    print(doc.vocab.strings[match_id], doc[start:end].text)
```

Run code Submit

OUTPUT

```
PATTERN1 Amazon Prime
PATTERN2 ad-free viewing
PATTERN1 Amazon Prime
PATTERN2 ad-free viewing
PATTERN2 ad-free viewing
PATTERN2 ad-free viewing
✓ Well done! For the token '-', you can match on the attribute 'TEXT',
'LOWER' or even 'SHAPE'. All of those are correct. As you can see, paying close
attention to the tokenization is very important when working with the
token-based 'Matcher'. Sometimes it's much easier to just match exact strings
instead and use the 'PhraseMatcher', which we'll get to in the next
exercise.
```

INPUT

```
import json
import spacy

with open("exercises/en/countries.json", encoding="utf8") as f:
    COUNTRIES = json.loads(f.read())

nlp = spacy.blank("en")
doc = nlp("Czech Republic may help Slovakia protect its airspace")

# Import the PhraseMatcher and initialize it
from spacy.matcher import PhraseMatcher

matcher = PhraseMatcher(nlp.vocab)

# Create pattern Doc objects and add them to the matcher
# This is the faster version of: [nlp(country) for country in COUNTRIES]
patterns = list(nlp.pipe(COUNTRIES))
matcher.add("COUNTRY", patterns)

# Call the matcher on the test document and print the result
matches = matcher(doc)
print([doc[start:end] for match_id, start, end in matches])
```

Run code  Submit

OUTPUT

```
[Czech Republic, Slovakia]
✓ Well done! Let's use this matcher to add some custom entities.
```

INPUT

```
import spacy
from spacy.matcher import PhraseMatcher
from spacy.tokens import Span
import json

with open("exercises/en/countries.json", encoding="utf8") as f:
    COUNTRIES = json.loads(f.read())
with open("exercises/en/country_text.txt", encoding="utf8") as f:
    TEXT = f.read()

nlp = spacy.load("en_core_web_sm")
matcher = PhraseMatcher(nlp.vocab)
patterns = list(nlp.pipe(COUNTRIES))
matcher.add("COUNTRY", patterns)

# Create a doc and reset existing entities
doc = nlp(TEXT)
doc.ents = []

# Iterate over the matches
for match_id, start, end in matcher(doc):
    # Create a Span with the label for "GPE"
    span = Span(doc, start, end, label="GPE")

    # Overwrite the doc.ents and add the span
    doc.ents = list(doc.ents) + [span]

    # Get the span's root head token
    span_root_head = span.root.head
    # Print the text of the span root's head token and the span text
    print(span_root_head.text, "-->", span.text)

# Print the entities in the document
print([(ent.text, ent.label_) for ent in doc.ents if ent.label_ == "GPE"])
```

Run code  Submit

OUTPUT

```
in --> Namibia
in --> South Africa
elections --> Cambodia
of --> Kuwait
as --> Somalia
Somalia --> Haiti
Haiti --> Mozambique
in --> Somalia
for --> Rwanda
Britain --> Singapore
War --> Sierra Leone
```

```
of --> Afghanistan
invaded --> Iraq
in --> Sudan
of --> Congo
earthquake --> Haiti
[('Namibia', 'GPE'), ('South Africa', 'GPE'), ('Cambodia', 'GPE'), ('Kuwait', 'G
PE'), ('Somalia', 'GPE'), ('Haiti', 'GPE'), ('Mozambique', 'GPE'), ('Somalia',
'GPE'), ('Rwanda', 'GPE'), ('Singapore', 'GPE'), ('Sierra Leone', 'GPE'), ('Afgh
anistan', 'GPE'), ('Iraq', 'GPE'), ('Sudan', 'GPE'), ('Congo', 'GPE'), ('Haiti',
'GPE')]
✓ Well done! Now that you've practiced combining predictions with
rule-based information extraction, you're ready for chapter 3, which will teach
you everything about spaCy's processing pipelines.
```